

- SAR Narrative Generator with Audit Trail
 - Abstract
 - System Architecture
 - Key Components
 - Methodology / Proposed System
 - A. Scalability & Performance
 - B. Security & Data Governance
 - C. The Audit Trail (Defensibility)
 - Tech Stack
 - Fraud Detection Model Design (Proposed)
 - 1. Model Architecture: Gradient Boosting (XGBoost)
 - 2. Feature Engineering Strategy
 - 3. Explainability (The "Why")
 - Future Scope
 - Any other comments on solution

SAR Narrative Generator with Audit Trail

Abstract

In the highly regulated banking sector, drafting Suspicious Activity Reports (SARs) is a critical yet resource-intensive task. Barclays, specifically, faces significant scrutiny following the **£42 million fine** in 2025 by the UK's FCA for "**poor handling**" of AML risks and a failure to act on "clear warning signs." Regulators have criticized the "one-and-done" approach to KYC, demanding continuous monitoring and defensible audit trails.

This proposal introduces an AI-driven SAR Narrative Generator that directly addresses these gaps using an **Integrated Audit Trail**. By automating the transition from "**Logic-Based Detection**" to "**Regulator-Ready Narrative**," the system not only reduces drafting time by 80% but ensures complete transparency. The solution solves the "black box" problem by citing specific data points and regulatory rules for every claim, proving *exactly why* a suspicion was formed. It leverages a dual-model architecture (Logic Engine + RAG-enhanced Llama 3.1) to produce defensible reports, avoiding future enforcement actions while freeing analysts to focus on complex investigations.

System Architecture

The system is built on a Modular Microservices architecture to ensure scalability and environment-aware deployment.

Key Components

1. Data Orchestration Layer (**data_ingestion.py**):

- Ingests siloed data including **KYC Profiles** (Identity), **Transaction Logs** (Behavior), and **Alert Context** (Trigger).
- Enriches raw alert data (e.g., **ALT-2024-001**) with customer details before processing.

2. Context & Knowledge Engine (RAG) (**rag_engine.py**):

- **Vector Database (ChromaDB)**: Stores high-dimensional embeddings of regulatory guidelines (e.g., FinCEN, FCA, FIU-IND) and standard SAR templates.
- **Retrieval Pipeline**: Uses LangChain to semantic search the **sar_guidelines** collection, ensuring the model references the most up-to-date laws.

3. Generative Core:

- A fine-tuned **Llama 3.1 (8B/70B)** synthesizes the enriched data and retrieved regulatory context to draft the narrative.
- Ensures "on-topic" generation using strict system prompts (see attached **prompt_template.md**).

4. Audit Logging & Explainability:

- A specialized service captures **Chain-of-Thought (CoT)** reasoning traces.
- Logs specific "Red Flag Indicators" (RFIs) triggered by the Logic Engine.

5. Human-in-the-Loop Interface (Streamlit):

- An interactive dashboard where compliance analysts review the drafted SAR.

- Allows for editing and "signing off," with all human edits captured to reinforce the model (RLHF).

Methodology / Proposed System

A. Scalability & Performance

- **Horizontal Scaling:** This system utilizes containerization (Docker/Kubernetes). Multiple instances of the generator can run in parallel, sharing the same ChromaDB vector store and PostgreSQL backend for high-volume batch processing.
- **Optimized Inference:** Utilizes **vLLM** for high-throughput serving of the Llama 3.1 model, ensuring reports are generated in seconds.
- **Efficient Retrieval:** `rag_engine.py` implements chunking strategies (500-token chunks) to optimize the relevance of retrieved regulatory context.

B. Security & Data Governance

- **Role-Based Access Control (RBAC):** Analysts can only see data relevant to their assigned cases, preventing cross-domain leakage (e.g., Private Banking vs. Retail).
- **Environment-Aware Logic:** The system detects deployment environments (AWS vs. On-Prem) to apply appropriate data residency and encryption rules.
- **PII Redaction:** Pre-processing layers redact sensitive PII from logs, ensuring only the generative model sees the full context needed for the report.

C. The Audit Trail (Defensibility)

The "Audit Trail" is a non-editable log stored in PostgreSQL that tracks:

- **Data Lineage:** specific transaction IDs and KYC fields used.
- **Regulatory Citations:** specific paragraphs from the RAG retrieval (e.g., "Per Section 314(b) of the USA PATRIOT Act...").
- **Reasoning Trace:** why the model classified the activity as suspicious.

Tech Stack

Category	Tool / Framework
LLM	Llama 3.1 (8B/70B) (Local/Bedrock)
Orchestration	LangChain / LangGraph
Vector Database	ChromaDB (Open Source) / AWS OpenSearch
Backend/Ingestion	Python (Pandas, FastAPI), <code>data_ingestion.py</code>
Frontend	Streamlit (Interactive Analyst Workbench)

| **Explore/Explain** | SHAP / LangChain Callbacks | | **Database** | PostgreSQL (Case storage & Audit Logs) |

Fraud Detection Model Design (Proposed)

To minimize false positives and provide granular reasons for suspicion, we propose a specialized Machine Learning layer before the Generative AI step.

1. Model Architecture: Gradient Boosting (XGBoost)

We selected **XGBoost** over deep learning for the core classification engine due to its superior performance on tabular financial data and native support for handling missing values.

- **Hyperparameters:** Optimized for `logloss` to output precise probabilities.
- **Imbalance Handling:** given that fraud is less than 1% of transactions, we employ **SMOTE (Synthetic Minority Over-sampling Technique)** during training to synthetically generate examples of fraud cases, preventing model bias towards the majority class.

2. Feature Engineering Strategy

The model inputs are derived from `data_ingestion.py` and `features.py`. Key features include:

- **Velocity Checks:** `velocity_1h`, `velocity_24h` (Count of transactions in rolling windows).
- **Behavioral Deviation:** `amount_zscore` (Standard deviations from the customer's 30-day moving average).
- **Pattern Risk:** `risk_rating_encoded` (Quantifying KYC risk levels).

3. Explainability (The "Why")

To meet the "Audit Trail" requirement, every prediction is passed through a **SHAP (SHapley Additive exPlanations)** explainer.

- **Output:** Instead of just a risk score (e.g., "0.92"), the system outputs: *"Risk Score 0.92 driven by: High Velocity (contribution +0.45) and New Beneficiary (contribution +0.30)."*
- **Integration:** These SHAP values are injected into the Llama 3.1 prompt, allowing the narratives to say, "The account is suspicious *because* of a rapid burst of small transactions," rather than a generic statement.

Future Scope

1. **Multi-Modal Analysis:** Expanding the generator to ingest images (e.g., check copies) or voice-to-text notes from investigator interviews.
2. **Cross-Border Synergy:** Automatically translating and re-formatting a UK SAR into an Indian STR format for global accounts, handling distinct taxonomy codes (e.g., XXVI vs. STR-A).
3. **Autonomous Feedback Loop:** Implementing **Reinforcement Learning from Human Feedback (RLHF)** where the model fine-tunes itself based on analyst edits to improve narrative "voice" and accuracy over time.

Any other comments on solution

To ensure the system is "Regulator-Ready," we have defined strict formats for the Prompt Engineering and Audit Logging.

- **Prompt Template:** We use a structured system prompt that forces the LLM to separate facts from analysis. (See [prompt_template.md](#))
- **Audit Logic:** The audit trail is structured as a JSON schema to allow for automated compliance checks. (See [audit_trail_schema.json](#))